COMSATS UNIVERSITY ISLAMABAD, LAHORE CAMPUS | DEPARTMENT OF COMPUTER SCIENCE

Database Systems: Master Study Guide (Lectures 1 to 15)

Instructor: Dr. Abid Sohail Bhutta (abidbhutta@cuilahore.edu.pk) | Textbooks: Hoffer et al. (11th Ed.) & Connolly & Begg (3rd/4th Ed.)

> **How to Use This Study Guide:** This guide condenses all 15 lecture slides into plain, easy-to-memorize explanations. Every lecture includes quick definitions, real-world analogies, comparison tables, clean diagrams, SQL examples, and solved self-assignments so you can grasp and recall every concept effortlessly during quizzes and exams.

Lecture 1: Introduction to Database, DBMS, Data, and Information

The primary objective of this course is twofold: first, to master **Database Modeling and Development** (designing blueprints for data), and second, to implement databases using **SQL in MS SQL Server**. Before building a database, you must understand what a database is and how it differs from the software that runs it.

1. Core Definitions: Data, Database, and DBMS

- **Data vs. Information:** *Data* consists of raw, isolated facts (such as the number "502" or the name "Ali"). *Information* is data that has been processed, organized, and given context so it is useful for decision-making (such as "Customer Ali is assigned to Agent 502").
- **Database (DB):** *"A shared collection of logically related data, and a description of this data, designed to meet the information needs of an organization."* Notice three key phrases in this definition:
  1. **Shared collection:** Multiple departments (Sales, Payroll, Inventory, Vendors, Accounting) use the same central repository rather than keeping separate files.
  2. **Logically related data:** Data is organized around real-world **Entities** (objects), **Attributes** (properties), and **Relationships** (connections).
  3. **Description of this data (System Catalog / Metadata):** A database does not just store raw rows; it also stores a self-describing dictionary of table structures and rules, which enables **program-data independence**.
- **Database Management System (DBMS):** *"A software system that enables users to define, create, maintain, and control access to the database."* If the database is a digital filing cabinet holding the data, the DBMS (such as MS SQL Server, Oracle, or MySQL) is the intelligent librarian that fetches, updates, and protects the files.

2. Components of a DBMS

- **Data Definition Language (DDL):** The sub-language used to define the database structure, specify data types, create tables, and set data constraints (for example, `CREATE TABLE`, `ALTER TABLE`).
- **Data Manipulation Language (DML):** The general inquiry and update facility (query language) used to read, insert, modify, and delete actual data rows (for example, `SELECT`, `INSERT`, `UPDATE`, `DELETE`).
- **Controlled Access Systems (The 4 Guards of a DBMS):**
  - **Security System:** Prevents unauthorized users from viewing or modifying data.
  - **Integrity System:** Enforces correctness rules so invalid data cannot enter the database.
  - **Concurrency Control System:** Manages simultaneous access when multiple users edit the same data at the same time.
  - **Recovery Control System:** Restores the database to a working state after a hardware or software crash.

3. Textbook Case Study: Property Rental System Schema

In Lecture 1 (Connolly & Begg), the Sales and Contracts departments both connect to a single DBMS, which manages four logically related tables:

```
PropertyForRent (propertyNo, street, city, postcode, type, rooms, rent, ownerNo)
PrivateOwner    (ownerNo, fName, lName, address, telNo)
Client          (clientNo, fName, lName, address, telNo, prefType, maxRent)
Lease           (leaseNo, propertyNo, clientNo, paymentMethod, deposit, paid,
rentStart, rentFinish)
```

Lecture 2: File Processing Systems vs. Database Management Systems

Before DBMS software existed, organizations used **Traditional File-Based Systems**. A file-based system is *"a collection of application programs that perform services for the end-users such as the production of reports. Each program defines and manages its own data."* Its core rule is: **One file, one application**.

1. Anatomy of an Invoice ("Stereos To Go" Example)

A single paper invoice from the company "Stereos To Go" actually contains four separate types of business data mixed together on one page: (1) **Invoice Header** (Invoice No, Date, Date Shipped), (2) **Customer Info** (Account No, Name, Address, City, State, Zip Code), (3) **Line Items Purchased** (Item No, Qty, Price), and (4) **Product & Manufacturer Details** (Product Code, Description). In a file-based system, this causes massive duplication across departments:

- **Customer Order File** (used by Invoice Program) stores: Invoice No, Account No, *Customer Name, Address, City, State, Zip*, Order Date, Product details.
- **Customer Account File** (used by Account Program) repeats: Account No, *Customer Name, Mailing Address, City, State, Zip*.
- **Customer Mailing List File** (used by Mailing Program) repeats again: *Customer Name, Mailing Address, City, State, Zip*.

2. Six Major Limitations of File-Based Systems

Limitation	Easy Explanation & Real-World Consequence
1. Separation & Isolation of Data	Because each program owns its own file, combining data from two files (such as matching Orders with Credit History) requires writing complex custom code.
2. Duplication of Data (Redundancy)	The same customer address is stored in three different files. This wastes disk space and human data-entry time.
3. Loss of Data Integrity	If a customer moves and updates their address in the Order file, the Mailing List file still has the old address. Data becomes inconsistent and untrustworthy.
4. Data Dependence	The exact physical structure of the file is hardcoded inside the application program (such as COBOL `FD Master-File` declarations). Changing a field length from 20 to 30 characters requires rewriting and recompiling every program.
5. Incompatible File Formats	Sales might write a file in COBOL while Accounting uses C or Python. Their file structures cannot easily talk to one another.
6. Fixed Queries	Reports are hardcoded. If a manager asks a new ad-hoc question, a programmer must write a brand-new program from scratch.

3. How the Database Approach Solves This: Data Abstraction

In COBOL file systems, the file structure is tightly bound inside the code (`05 ID PIC X(10)` , `05 Customer-Fname PIC X(25)`). A DBMS introduces **Data Abstraction**: a clean separation between the data's structure (stored in the central repository's metadata) and the application programs. The DBMS acts as a **Single Access Point** serving a multitude of applications.

4. Five Components of a Database Environment

Memory Hook (H-S-D-P-P): *"Hardware And Software Drive People's Procedures."*

1. **Hardware:** Physical computers, servers, network devices, and storage disks.
2. **Software:** The DBMS itself, operating system, application programs, and query tools.
3. **Data:** The facts stored in the central repository, organized into a global **Schema** and partitioned into user **Subschemas**.
4. **Procedures:** The instructions, policies, and rules that govern how the database is designed, accessed, backed up, and used.
5. **People:** Data Administrators (DA, who manage data policies), Database Administrators (DBA, who manage technical DBMS operations), Database Designers (Logical and Physical), Application Developers, and End-Users (both novice and high-powered users).

5. Advantages vs. Disadvantages of the Database Approach

Advantages of Database Approach (14 Points)	Disadvantages of Database Approach (9 Points)
1. Control of data redundancy 2. Data consistency (single source of truth) 3. Greater informational gain from the same data 4. Data sharing as an organizational resource 5. Improved data integrity, validity, and consistency 6. Improved access and security 7. Enforcement of organizational standards 8. Economy of scale (centralization & consolidation) 9. Balancing of conflicting requirements via the DBA 10. Improved data accessibility and responsiveness 11. Increased programmer productivity 12. Improved maintenance via data independence 13. Increased concurrency (multi-user access) 14. Improved backup and recovery services	1. High software complexity 2. Large size (requires significant disk and memory) 3. High licensing cost of commercial DBMS 4. Dedication of technology and human resources 5. Additional hardware upgrade costs 6. High cost of converting legacy file systems 7. Performance overhead (general-purpose DBMS can be slower than a custom single-purpose file for simple tasks) 8. Higher impact of failure (centralized failure affects everyone) 9. Processing can be slow in heavy production environments if poorly tuned

Lecture 3: Data Flow Diagrams (DFDs) & Process Decomposition

A **Data Flow Diagram (DFD)** is a visual blueprint showing how data moves through an information system: where data originates, what processes transform it, where it is stored, and where it goes.

1. The Four DFD Symbols & Naming Guidelines

DFD Component	Visual Shape	Naming Rule	Examples (Lemonade Stand)
External Entity (Source/Sink)	Rectangle / Square	**Noun** (person, department, or outside system)	`CUSTOMER`, `VENDOR`, `EMPLOYEE`
Process	Circle or Rounded Rectangle	**Verb Phrase** (action) or System/Subsystem Name	`0.0 Lemonade System`, `1.1 Record Order`, `3.3 Pay Vendor`
Data Store	Open-ended Rectangle or Parallel Lines	**Noun** (plural or collective data repository)	`ORDER`, `PAYMENT`, `RAW MATERIALS`, `INVENTORY`

DFD Component	Visual Shape	Naming Rule	Examples (Lemonade Stand)
Data Flow	Arrow line connecting components	**Name of Data** (Noun phrase)	`Customer Order`, `Sales Forecast`, `Time Worked`

2. Five Steps to Construct a DFD (Lemonade Stand Walkthrough)

1. **Step 1: Create a list of business activities and group them into logical functional areas.**
   - *Sales Group:* Customer Order, Collect Payment (and Sales Forecast).
   - *Production Group:* Serve Product, Produce Product, Store Product.
   - *Procurement Group:* Order Raw Materials, Receive Items, Pay for Raw Materials.
   - *Payroll Group:* Record Time Worked, Calculate Payroll, Pay for Labor.
2. **Step 2: Construct the Context Level DFD (Process 0.0).** Draw a single central process (`0.0 Lemonade System`) surrounded only by External Entities (`CUSTOMER` , `EMPLOYEE` , `VENDOR`) and the main input/output data flows. *Never draw Data Stores on a Context Diagram.*
3. **Step 3: Construct the Level 0 DFD.** Zoom inside `0.0` to show the major logical subsystems: `1.0 Sale` , `2.0 Production` , `3.0 Procurement` , and `4.0 Payroll` , and how data flows between them.
4. **Step 4: Construct Level 1 to n DFDs.** Break each Level 0 subsystem into detailed atomic processes (such as `1.1` , `1.2` , `1.3`) and introduce the actual **Data Stores** (files/tables) they read from and write to.
5. **Step 5: Check against DFD rules** (every process must have at least one input and one output; data cannot flow directly from an External Entity to a Data Store without passing through a Process).

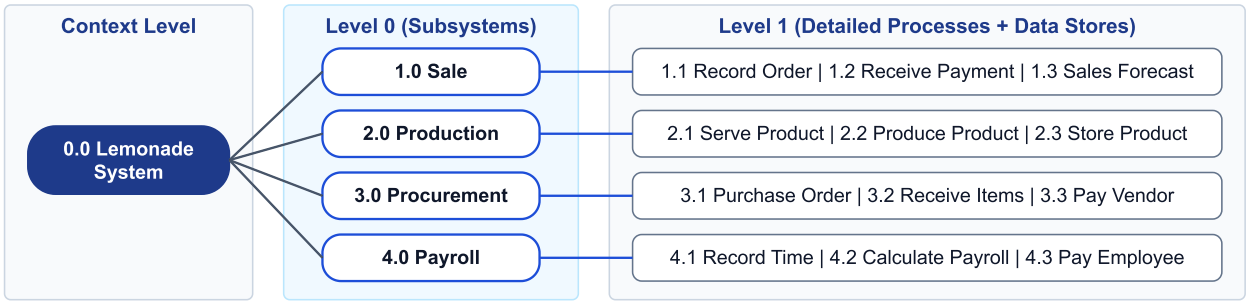


Figure 1: Process Decomposition from Context Level (0.0) to Level 0 and Level 1 DFDs (Lemonade Stand).

Lecture 4: Bus Garage DFD Example, Schemas, and Three-Level Architecture

1. DFD Case Study 2: Bus Garage Repairs

Scenario Summary: Buses come to a garage for repairs. A mechanic and helper perform the repair, record the reason for the repair, and record the total cost of all parts used on a Shop Repair Order. Information on labor, parts, and repair outcome is used for billing by Accounting, parts monitoring by the Inventory Management System, and performance reviews by the Supervisor.

- **External Entities (6):** `Bus` (sends mechanical problem to be repaired; receives fixed mechanical problems), `Helper` (sends labor), `Mechanic` (sends labor), `Supervisor` (receives repair summary), `Inventory Management System` (receives list of parts used), `Accounting` (receives labor and parts cost details).
- **Central System (Context Process 0.0):** `Bus Repair Process System`.
- **Level 0 Subprocesses (5):** (1) Record Bus ID and reason for repair, (2) Determine parts needed, (3) Perform repair, (4) Calculate parts extended and total cost, (5) Record labor hours and cost.
- **Data Stores (4):** `Personnel file`, `Repairs file`, `Bus master list`, `Parts list`.
- **Data Flows (5):** `Repair order`, `Bus record`, `Parts record`, `Employee timecard`, `Invoices`.

2. Schema vs. Subschema

- **Schema:** The complete, global logical catalog of *all* data and relationships retained in the entire database.
- **Subschema:** A subset or individual customized **View** of the database tailored to specific users or application programs so they only see what is relevant to their job.

3. The ANSI-SPARC Three-Level Architecture

To keep users insulated from low-level disk details, modern database systems are divided into three distinct architectural levels. **Memory Hook (E-C-I):** *"External Users View, Conceptual Designers Plan, Internal Disks Store."*

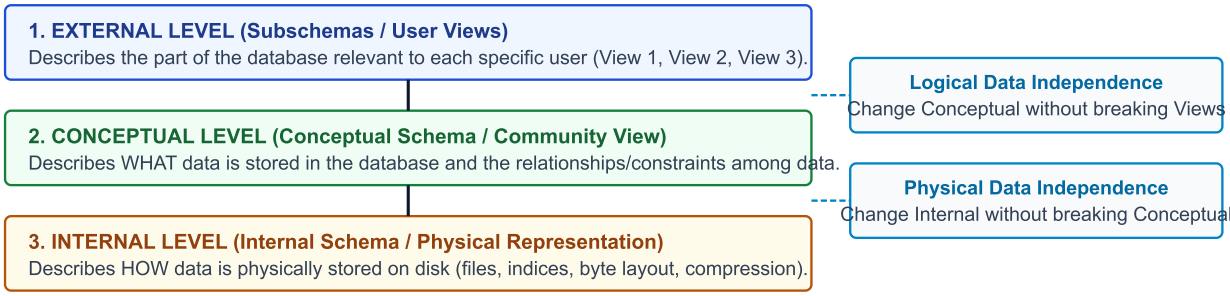


Figure 2: The Three-Level ANSI-SPARC Architecture and the Two Types of Data Independence.

Lecture 5: Data Independence, Evolution of DB Systems, and Data Models

1. Logical vs. Physical Data Independence ("Plug and Play")

- **Logical Data Independence:** The immunity of the **External Schemas (User Views)** to changes in the **Conceptual Schema**. For example, if the DBA adds a new table or a new column to the conceptual schema, existing user views and application programs continue working without any modification.
- **Physical Data Independence:** The immunity of the **Conceptual Schema** to changes in the **Internal Schema**. For example, if the DBA moves data to a new hard drive, changes the file organization from sequential to B-Tree, or creates a new index, the conceptual table definitions and user queries remain completely untouched.

2. Historical Evolution of Database Systems

Era / Timeline	Database Generation / Paradigm	Key Characteristic
1960s to 1980s	Flat File Systems	One file per program (COBOL/FORTRAN); high redundancy, no data independence.
1970s to 1990s	Hierarchical & Network DBMS	Tree structures (1:M only, IBM IMS) and Pointer Networks (CODASYL DBTG).
1980s to Present	Relational DBMS & Data Warehousing	Data stored in mathematical tables (rows/columns) queried via SQL.
1990s to Present	Object-Oriented, Object-Relational & Web DBs	Stores complex objects/classes and supports internet-scale applications.
2010s to Present	NoSQL, JSON & XML Databases	Flexible document, key-value, and graph models for unstructured big data.

3. Data Models and Their Three Components

A **Data Model** is an integrated collection of concepts for describing and manipulating data, relationships between data, and constraints on the data in an organization. Every data model has three parts (**Memory Hook: S-M-I**):

1. **Structural Part:** A set of rules applied to how the database is constructed (such as tables, trees, or objects).
2. **Manipulative Part:** Defines the types of operations allowed on the data (such as searching, updating, or joining).
3. **Integrity Rules:** Rules that ensure the accuracy and validity of the data (such as primary and foreign key rules).

Database models are grouped into two broad families: **Conceptual Models** (such as the Entity-Relationship Model and Object-Oriented Model, which focus on *what* data is represented and serve as design blueprints supporting 1:1, 1:M, and M:N relationships) and **Implementation Models** (Hierarchical, Network, Relational, and Object-Oriented, which focus on *how* data is stored in the DBMS).

Lecture 6: Implementation Database Models (Hierarchical & Network)

1. Hierarchical Database Model (HDBM)

- **Core Structure:** Logically represented by an **upside-down tree**. A single table/segment acts as the **Root Segment** at the top, from which Level 1, Level 2, and Level 3 child segments branch downward.
- **Golden Rule of HDBM:** A parent segment can have many children (**1:M relationship**), but **each child segment can have ONLY ONE parent**. Parents and children are connected by physical **pointers**.
- **Tree Traversal Order:** Uses a *left-list hierarchical path* (preorder traversal). In the filing cabinet assembly example: `Final Assembly (Root) -> Component A -> Assembly A -> Part A -> Part B -> Component B -> Component C -> Assembly B -> Part C -> Part D -> Part E -> Assembly C`.
- **Classic Use Case:** Mainframe **Banking Systems** (IBM IMS), where one customer account (parent) has thousands of fixed debit/credit transactions (children) in a strict 1:M hierarchy.
- **Pros vs. Cons:** Simple hierarchy, uniform security, high speed for fixed 1:M transactions; however, physical pointer implementation is complex, it lacks structural independence, and it cannot directly represent M:N relationships without data duplication.

2. Network Database Model (NDBM)

- **Core Structure:** Standardized by the **Database Task Group (DBTG)** of CODASYL. It overcomes the main flaw of the hierarchical model by allowing **each child record to have MULTIPLE parent records**.
- **Set Terminology:** Instead of "parent and child", NDBM uses set theory: each relationship is called a **Set** consisting of an **Owner Record** (parent) and a **Member Record** (child). A member record can belong to multiple sets from different owners.
- **Slide Example:** An `INVOICE` record is a member of two sets at once: the *Commission set* (owned by `SALESREP`) and the *Sales set* (owned by `CUSTOMER`). Similarly, `INV_LINE` is owned by both `PRODUCT` (*Inventory set*) and `INVOICE` (*Line set*).
- **Three DBTG Components:** (1) *Network Schema* (global organization), (2) *Subschema* (application view), and (3) *Database Management Language* (Schema DDL, Subschema DDL, and DML).
- **Pros vs. Cons:** Handles complex multi-parent relationships and enforces standards, but suffers from extreme system complexity ("designed by programmers for programmers, not end-users") and lacks structural independence (changing pointer sets breaks application code).

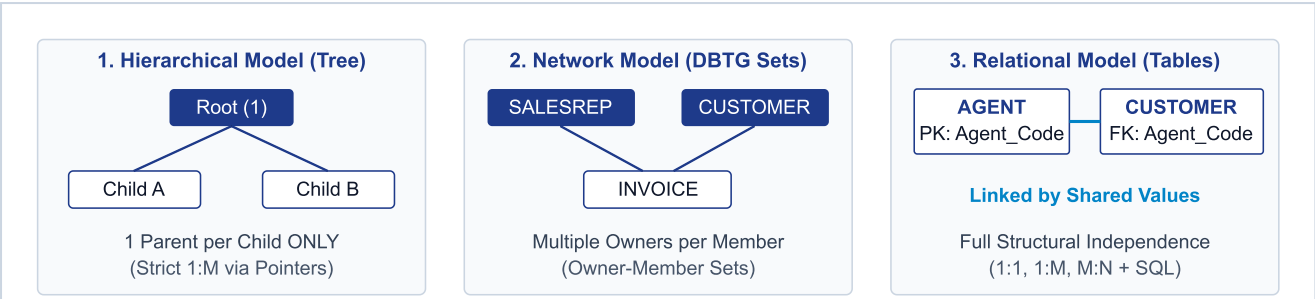


Figure 3: Structural Comparison of Hierarchical, Network, and Relational Database Models.

Lecture 7: Object-Oriented Data Model (OODB) and Relational Data Model (RDB)

1. Object-Oriented Database Model (OODB / ODBMS)

An **Object-Oriented Database (OODB)** stores data in the form of **Objects**, which are instances of **Classes**. Unlike flat relational rows, an object bundles both data properties (*variables/attributes*) and behavior (*messages/methods*) or nested sub-objects into a single unit.

- **C++ Class Style Example (Slide 6):** A `class CLERK` encapsulates variables (`char name; string address; int id; int salary;`) alongside messages/methods (`get_name()`, `get_address()`, `annual_salary()`).
- **JSON / Document Object Style Examples (Slides 7 to 9):** Modern document databases (such as MongoDB) store rich hierarchical objects containing nested objects and arrays in a single record, such as a `book` object containing an array of `authors: ["Kristina Chodorow", "Mike Dirolf"]` and a nested `publisher: { name: "O'Reilly Media", founded: "1980", location: "CA" }`.
- **OODB Pros & Cons:** Excels at real-world modeling, model simplicity for complex nested structures, and reduced data size; however, it historically suffered from a lack of a universal SQL query standard and lack of built-in automatic relational constraints.

2. Relational Database Model (RDBM)

Introduced by E.F. Codd in 1970, the **Relational Model** lets users and designers work in a natural human logical environment. Users see the database simply as a collection of **Tables (Relations)** made of row/column intersections, while the RDBMS hides all physical disk storage details.

- **How Tables Connect:** Instead of physical disk pointers, tables are linked logically by sharing common attributes. In Lecture 7's example, the `AGENT` table (`AGENT_CODE`, `AGENT_LNAME`, `AGENT_PHONE`) connects to the `CUSTOMER` table (`CUS_CODE`, `CUS_LNAME`, ..., `AGENT_CODE`) because both share `AGENT_CODE` (for example, Agent 502 serves Customers 10010, 10012, 10013, and 10016).
- **Key Advantage (Structural Independence):** Because data access paths are completely irrelevant to the logical table design, changing how data is stored physically never breaks queries or applications. Combined with **SQL (a 4th Generation Language / 4GL)**, users can write ad-hoc queries effortlessly.

- **Disadvantages:** Substantial hardware/software overhead to hide physical details; makes it easy for untrained users to create poorly designed tables; and can lead to "islands of information" if departments create isolated databases.

3. Master Classification of All Data Models

- **Object-Based Data Models:** Entity-Relationship (ER), Semantic, Knowledge-based, Functional, Object-Oriented.
- **Record-Based Data Models (Transaction-oriented):**
  - *Relational* (Oracle, DB2, Sybase, Informix, MS SQL Server, Ingres): Uses mathematical relations (Tables, Rows, Columns).
  - *Network* (CODASYL DBTG): Uses Record types and Data items linked in sets.
  - *Hierarchical* (IBM IMS): Uses Segment types and Fields in trees. (Compare with COBOL: Files, Records, Fields).
- **Physical Data Models (How bytes are stored):** Unifying model, Frame memory.

Lecture 8: Functions of a DBMS, DBMS Architecture, and Metadata

1. The 10 Classic Functions of a DBMS

Memory Hook (D-U-T-C-R-A-C-I-D-U): Every full-featured DBMS performs these 10 core services:

1. **Data storage, retrieval, and update:** The fundamental ability to save and fetch data.
2. **User-accessible catalog:** Provides a system catalog (data dictionary) storing metadata.
3. **Transaction support:** Ensures all-or-nothing execution of logical units of work.
4. **Concurrency control services:** Prevents data corruption when multiple users update records simultaneously.
5. **Recovery services:** Restores the database after system failures.
6. **Authorization services:** Enforces security and user permissions.
7. **Support for data communications:** Allows remote client applications to talk to the server over a network.
8. **Integrity services:** Enforces primary key, foreign key, and domain rules.
9. **Services to promote data independence:** Supports views and separates logical schemas from physical files.
10. **Utility services:** Provides tools for data import/export, monitoring, index rebuilding, and backup.

2. DBMS Architecture & Database Manager Internals

When a Programmer, End-User, or DBA interacts with a DBMS, their request flows through specialized layers before reaching the operating system's disk buffers:

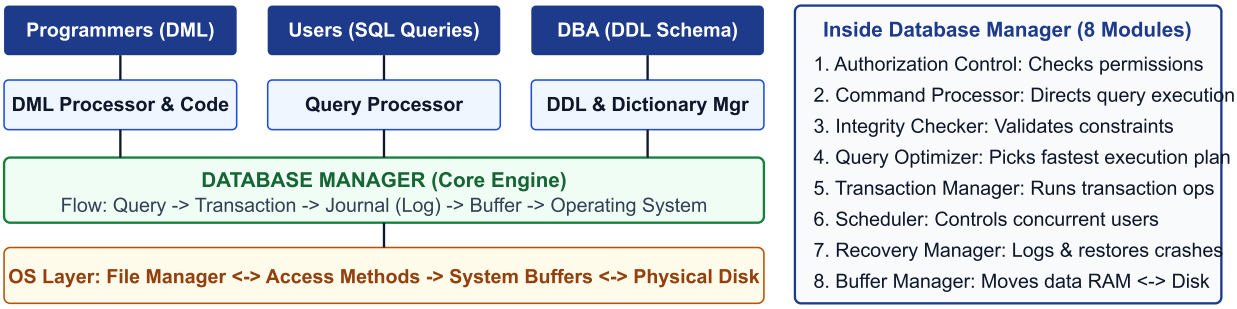


Figure 4: End-to-End DBMS Architecture and the 8 Internal Modules of the Database Manager.

3. Metadata & The System Catalog (Data Dictionary)

Metadata is "data about data", or the structural information that leads the DBMS to the actual user data. When a user runs `SELECT * FROM tables`, the DBMS first consults the **Data Dictionary (System Catalog)** to verify that the table exists, check user permissions, and find its column types.

- **Four Types of Metadata (Slide 12):**
  1. **Technical Metadata:** Database system names, table names, table sizes, column data types (such as `int`, `nvarchar(50)`, `date`), attributes, primary keys, foreign keys, and indices.
  2. **Business Metadata:** Data ownership, changing organizational policies, business rules, and regulations related to the enterprise.
  3. **Operational Metadata:** Information describing operational execution, policies, currency of data, and governance rules.
  4. **Descriptive Metadata:** Details describing a file, folder, book, image, or video (such as title, author name, file size, and publication date).

Lecture 9: Relational Data Model, Properties of Relations, and Keys

1. Anatomy and Mathematical Properties of a Relation

In the Relational Model, SQL acts as a set-oriented language where *Database + Criteria -> Data Set + Presentation Method -> Information*. Every table (formal name: **Relation**) has formal anatomical terms:

- **Entity / Relation:** The table itself (such as `Employee` or `Persons`).
- **Attributes (Columns):** Named vertical columns of the table. The **Degree of a relation** is the total number of attributes (columns).
- **Tuples (Rows):** Horizontal records in the table. The **Cardinality of a relation** is the total number of tuples (rows).
- **Domain:** The set of allowable values (data type, format, and range) that an attribute is permitted to assume.

Seven Strict Properties of a Valid Relation: (1) Distinct relation name, (2) Each cell contains exactly **one atomic (single) value** (no repeating groups), (3) Distinct attribute names, (4) All values in a column come from the same domain, (5) Order of columns has no significance, (6) Each tuple is distinct (no duplicate rows), and (7) Order of rows has no significance.

2. Complete Hierarchy of Relational Keys

Because duplicate rows are forbidden in a relation, we need a reliable way to uniquely identify every tuple. Think of keys as a funnel narrowing from broadest (Super Key) to most specific (Primary Key):

Key Type	Clear Definition & Key Rule	Example in Relation R(A, B, C, D, E)
Super Key	Any attribute or combination of attributes that uniquely identifies a tuple. *May contain extra, unnecessary columns.*	Given `A -> BCDE` and `BC -> ADE`: Super Keys include `A`, `BC`, `ABC`, `AD`, `ABCDE`.
Candidate Key (SQL `UNIQUE`)	A **minimal** Super Key: uniquely identifies the tuple (**Uniqueness**) and contains *no proper subset* that is also a super key (**Irreducibility**).	Only `A` and `BC` are Candidate Keys. (`AD` is not minimal because `A` alone is already unique).
Primary Key (PK)	The ONE Candidate Key chosen by the designer to identify tuples in the table. Must be **NOT NULL**, efficient, and **never change** over the life of the entity.	`A` is chosen as the Primary Key because a single column is more efficient than a two-column key (`BC`).
Composite Key	A key (Candidate, Primary, or Foreign) made up of **two or more attributes** working together.	`BC` is a Composite Candidate Key; `PRIMARY KEY (ID, LastName)` is a Composite PK.
Surrogate Key	An **artificial, system-generated numeric key** (auto-incremented: 1, 2, 3...) used instead of a natural business key.	`Personid INT IDENTITY(1,1) PRIMARY KEY` (SQL Server) or `AUTO_INCREMENT` (MySQL).

3. Surrogate Keys: When to Use, Pros, and Cons

- **When to use a Surrogate Key:** To replace a bulky Composite Primary Key, when a natural key might change or be used inappropriately, or in denormalized data warehouse tables.
- **Advantages:** Eliminates composite key complexity; fast integer indexing; handles multiple historical versions of data easily.
- **Disadvantages:** The number has zero business meaning; users can accidentally insert duplicate real-world people with different auto-generated IDs if the natural attributes are not protected by a `UNIQUE` constraint.

```
-- SQL Server Surrogate Key Syntax
CREATE TABLE Persons (
    Personid INT IDENTITY(1,1) PRIMARY KEY,
    LastName VARCHAR(255) NOT NULL,
    FirstName VARCHAR(255),
    Age INT
);

-- Table-Level Composite Primary Key Syntax
CREATE TABLE Persons_Composite (
    ID INT NOT NULL,
    LastName VARCHAR(255) NOT NULL,
    FirstName VARCHAR(255),
    Age INT,
    CONSTRAINT PK_Person PRIMARY KEY (ID, LastName)
);
```

4. Solved Self-Assignment (Lecture 9, Slide 16)

- **Q1: How to change the value of a Primary Key for an entity (row)?**
  Answer: Use an `UPDATE` statement: `UPDATE Persons SET ID = 105 WHERE ID = 101;`. Note that good primary keys should remain constant, and if child tables reference this key via a Foreign Key, the FK must be configured with `ON UPDATE CASCADE`.
- **Q2: How to change the Primary Key definition for an existing table?**
  Answer: First drop the existing PK constraint, then add the new one: `ALTER TABLE Persons DROP CONSTRAINT PK_Person;` followed by `ALTER TABLE Persons ADD CONSTRAINT PK_Person PRIMARY KEY (ID);`.

Lecture 10: Relational Integrity Rules and SQL Constraints

1. The Three Relational Integrity Rules

1. **Entity Integrity (Rule for Primary Keys):** In a base relation, **no attribute of a Primary Key can ever be NULL**. Even if a Primary Key is a composite of three columns, none of the three columns can be NULL, because every row must be uniquely identifiable.
2. **Referential Integrity (Rule for Foreign Keys):** A **Foreign Key (FK)** is an attribute in a child table that references a Candidate/Primary Key in a parent (home) table. Referential integrity states that a Foreign Key value must either:
   - **Match an existing key value** in the parent table (for example, in the Office Meeting Room diagram, `FIN-MG`, `SAL-MG`, and `MAR-MG` exist in `userinfo`, so they are allowed into `meeting_details`, whereas `INV-MG` does not exist in `userinfo` and is blocked!), OR
   - Be **wholly NULL** (if the relationship is optional and the column allows NULLs).
3. **Enterprise Constraints (Business Rules):** Additional organization-specific rules enforced on the database, such as "Every employee must be 18 or older" or "A professor can teach at most 4 classes".

2. Summary of SQL Constraints

Constraints can be written at the **Column Level** (inline right after a column's data type) or at the **Table Level** (at the bottom of `CREATE TABLE`, which is required whenever a constraint spans multiple columns).

- `NOT NULL`: Forbids unknown or empty (NULL) values in a column.
- `UNIQUE`: Ensures every value in the column is distinct.
- `PRIMARY KEY`: Combines `NOT NULL` + `UNIQUE` to identify each row.
- `FOREIGN KEY`: Links a child column to a parent table's key (`REFERENCES ParentTable(Column)`).
- `CHECK`: Validates that values satisfy a boolean expression (`CHECK (Age >= 18)`).
- `DEFAULT`: Automatically inserts a fallback value if the user omits the column (`City VARCHAR(255) DEFAULT 'Sandnes'`).

```
CREATE TABLE Person (
    PersonId INT,
    FirstName NVARCHAR(40) NOT NULL,
    LastName NVARCHAR(40) NOT NULL,
    Age INT CHECK (Age >= 18),
    City VARCHAR(255) DEFAULT 'Sandnes',
    CONSTRAINT PK_Person PRIMARY KEY (PersonId)
);

CREATE TABLE Orders (
    OrderID INT NOT NULL,
    OrderNumber INT NOT NULL,
    PersonID INT,
    PRIMARY KEY (OrderID),
    FOREIGN KEY (PersonID) REFERENCES Person(PersonId)
);
```

3. Solved Self-Assignment (Lecture 10, Slide 12)

- **Q1: Can an attribute other than a Primary Key (PK) be used as a Foreign Key target?**
  Answer: Yes! A Foreign Key in a child table can reference any **Candidate Key** (any column or set of columns with a `UNIQUE` constraint) in the parent table, not exclusively the `PRIMARY KEY`.
- **Q2: How do you drop (delete) a tuple in a parent table when it is being referenced in a child table via a Foreign Key?**
  Answer: By default, SQL aborts the deletion to protect referential integrity. To delete the parent row, you must either (a) first delete or update the referencing child rows in the child table, or (b) define the Foreign Key constraint with `ON DELETE CASCADE` (which automatically deletes the child rows) or `ON DELETE SET NULL` (which sets the child's FK column to NULL).
- **Q3: What is the minimum Foreign Key eligibility criteria for a column?**
  Answer: (1) The referenced parent column must have a `PRIMARY KEY` or `UNIQUE` constraint, and (2) the child FK column and parent column must share the **exact same data type, length, and domain**.

Lecture 11: SQL Views (Subschemas) and SQL Indexes

1. SQL Views: Virtual Tables for the External Level

While a **Base Relation** is a physical table whose rows are stored on disk, a **View** is a **virtual relation** (subschema) whose contents are dynamically derived by running a saved `SELECT` query over one or more base relations. A standard view takes up almost no disk space because only its query definition is stored in the Data Dictionary.

- **Why Use Views? (5 Key Benefits):** (1) **Security:** Hides sensitive columns like `Salary` from general staff, (2) **Customization:** Tailors the table layout to each department's needs, (3) **Simplicity:** Wraps complex multi-table joins into a simple virtual table name, (4) **Logical Data Independence:** Shields user applications from changes to base tables, and (5) **Zero Data Duplication** (except Materialized Views).

View Type	No. of Base Tables	Functions / GROUP BY?	DML (INSERT/UPDATE) Allowed?	Key Purpose & Storage
1. Simple View	One table only	No	**Yes**	Filters rows/columns of a single table (`WHERE DeptID = 2`). Virtual (stores query only).
2. Complex View	One or more tables	**Yes** (e.g., `AVG()`)	**Not Allowed**	Combines joins and grouped aggregations (`GROUP BY DeptID`). Virtual (stores query only).
3. Inline View	Subquery in `FROM`	Optional	N/A (Query-scoped)	A temporary derived table written inside the `FROM (SELECT ...)` clause of a larger query.
4. Materialized View (Snapshot)	One or more tables	Yes	Read-only snapshot (must be refreshed)	**Physically stores retrieved data on disk.** Ultra-fast access; heavily used in **Data Warehousing**.

2. SQL Indexes: B-Tree Pointers for the Physical Level

An **Index** is a physical schema object that speeds up row retrieval using sorted pointers, just like the alphabetical topic index at the back of a textbook lets you jump straight to page 245 without scanning every page from page 1.

- **The Golden Trade-Off of Indexes:** Indexes **dramatically speed up** read queries (`SELECT` , `WHERE` , `JOIN` , `ORDER BY`) by reducing disk I/O, but they **slow down write operations** (`INSERT` , `UPDATE` , `DELETE`) because every time a row changes, the DBMS must also reorganize the index tree.

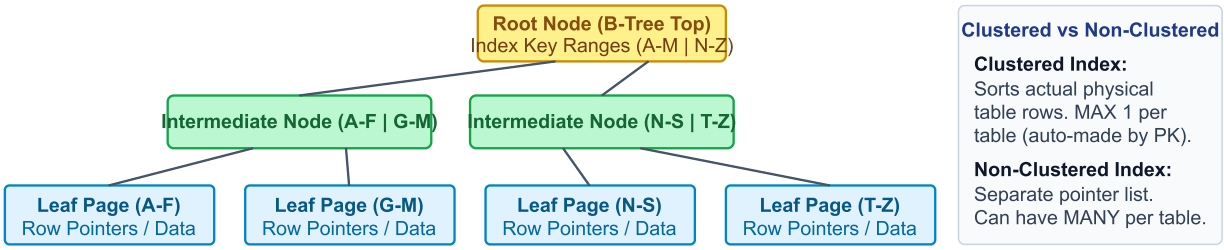


Figure 5: B-Tree Index Hierarchy (Root -> Intermediate -> Leaf Pages) and Clustered vs. Non-Clustered Rules.

```
-- Creating a View and an Index in SQL
CREATE VIEW emp_view AS
SELECT EmployeeID, Ename FROM Employee WHERE DeptID = 2;

CREATE INDEX IndexCustomerName
ON Customer_2 (LastName ASC, FirstName ASC);
```

Lectures 12 & 13: Conceptual Schema Design & ER Diagram Fundamentals

1. The 7 Steps of Database Schema Design

Memory Hook (R-C-S-L-U-P-S): (1) **Requirements Analysis** (user needs), (2) **Conceptual Design** (high-level ER Diagram), (3) **Schema Refinement** (normalization and consistency checks), (4) **Logical Design** (translating ERD into relational tables), (5) **User Interfaces Design** (external views), (6) **Physical Design** (indexes and disk layout), and (7) **Security Design** (access permissions).

2. All 6 Types of Attributes & Entity Types in ER Diagrams

ERD Concept	Visual ERD Notation	Easy Definition & Lecture Example
1. Simple Attribute	Single solid oval	Atomic value that cannot be divided further (`Age` , `Gender` , `Class`).
2. Composite Attribute	Oval branching into smaller ovals	Made of multiple smaller simple sub-attributes (`Address` -> `Street` , `City` , `State` , `Country` ; `Name` -> `F_name` , `M_name` , `L_name`).
3. Single-Valued Attribute	Single solid oval	Holds only one value per entity instance (`DOB` , `Roll_no`).
4. Multi-Valued Attribute	**Double-line oval**	Can hold multiple values for a single person (`Mob_no` , `Email_id`).
5. Derived Attribute	**Dashed-line oval**	Calculated dynamically from another attribute (`Age` derived from `DOB`).
6. Key Attribute	Oval with **underlined text**	Uniquely identifies an entity (solid underline: `ssn` ; dashed underline: weak partial key `pname`).

ERD Concept	Visual ERD Notation	Easy Definition & Lecture Example
Strong Entity Set	Single-line rectangle	Has its own Primary Key and exists independently (`Employees` , `Student`).
Weak Entity Set	**Double-line rectangle + Double-line diamond**	Cannot be uniquely identified without an Owner Entity's PK + its own Partial Key/Discriminator (`Dependents` owned by `Employees` via `Policy`).

3. Relationship Degree, Cardinality Notations, and Participation

- **Degree of a Relationship (Number of participating entity sets):**
  - **Unary (Recursive / Self-Relationship) [Degree = 1]:** One entity set relates to itself! Example: `employee` *works-for* `employee` with role labels `manager` and `worker` on the lines (or `COURSE` *requires* `COURSE`).
  - **Binary Relationship [Degree = 2]:** Connects **two** entity sets (such as `Employees` *Works_In* `Departments`).
  - **Ternary Relationship [Degree = 3]:** Simultaneously connects **three** entity sets: for example, `Employees` , `Departments` , and `Locations` linked by `Works_In2` (recording which city office an employee works at for a department), or `Works_In3` adding `Duration (from, to)` so an employee can work in a department over multiple distinct periods.
- **Decoding Arrow Notation vs. Crow's Foot Notation (Crucial Exam Tip!):**
  - **Textbook Arrow Notation (Korth / Ramakrishnan slides):** A **directed arrow (`->`)** pointing to an entity or diamond means **"At most One (1)"**. An undirected line (`---`) means **"Many (M)"**. Thus, `customer <-- borrower --- loan` is **1-to-Many** (one customer has many loans); `customer --- borrower --> loan` is **Many-to-1**; `customer --- borrower --- loan` is **Many-to-Many**; and `Employees --- Manages <-- Departments` means each department has **at most one manager**.
  - **Crow's Foot / Fork Notation (Lectures 12, 14, 15):** A three-pronged **fork / crow's foot** on an entity means **"Many (M)"**, while a plain line or single bar means **"One (1)"**.
- **Participation Constraints (Single Line vs. Double Line):**
  - **Partial Participation (Optional):** Shown by a **single line**. Not every entity must participate (for example, a newly created `Course` may have zero enrolled students).
  - **Total Participation (Mandatory):** Shown by a **double line (`==`)**. Every single entity in the set *must* participate in the relationship (for example, every `Loan` must belong to at least one `Customer` , and every Weak Entity must have total participation with its Owner).

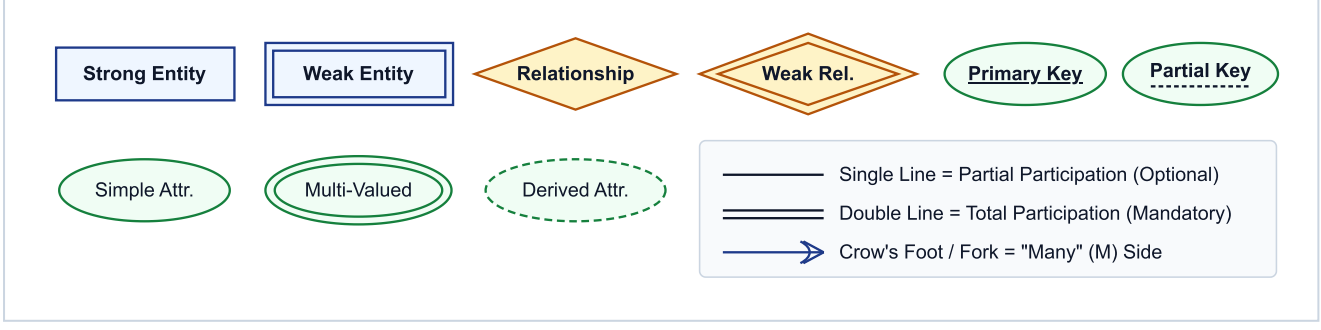


Figure 6: Visual Reference Guide for All ERD Symbols, Attributes, Entities, and Participation Lines.

4. SQL Translation of Ternary Relationships & Weak Entities

```
-- 1. Translating a Ternary Relationship (Employees + Departments + Locations)
CREATE TABLE Works_In2 (
    ssn   CHAR(11),
    did   INTEGER,
    city  CHAR(20),
    since DATE,
    PRIMARY KEY (ssn, did, city),
    FOREIGN KEY (ssn)  REFERENCES Employees,
    FOREIGN KEY (did)  REFERENCES Departments,
    FOREIGN KEY (city) REFERENCES Locations
);

-- 2. Translating a Weak Entity Set (Dependents + Policy merged into 1 table)
CREATE TABLE Dep_Policy (
    pname CHAR(20),
    age   INTEGER,
    cost  REAL,
    ssn   CHAR(11),
    PRIMARY KEY (pname, ssn),
    FOREIGN KEY (ssn) REFERENCES Employees ON DELETE CASCADE
);
```

Lectures 14 & 15: Step-by-Step ERD Construction & Schema Refinement

1. How to Build an ERD from a Problem Statement (University Case Study)

Golden Rule for Reading Scenarios: Nouns become *Entities* or *Attributes*, and **Verbs** become *Relationships*. Let us trace the Lecture 14 University specification:

*"A university consists of a number of **departments**. Each department **offers** several **courses**. A number of **modules make up (includes)** each course. **Students enrol in** a particular course and **take** modules towards the completion of that course. Each module is **taught by** a **lecturer** from (**employs**) the appropriate department, and each lecturer **tutors** a group of students."*

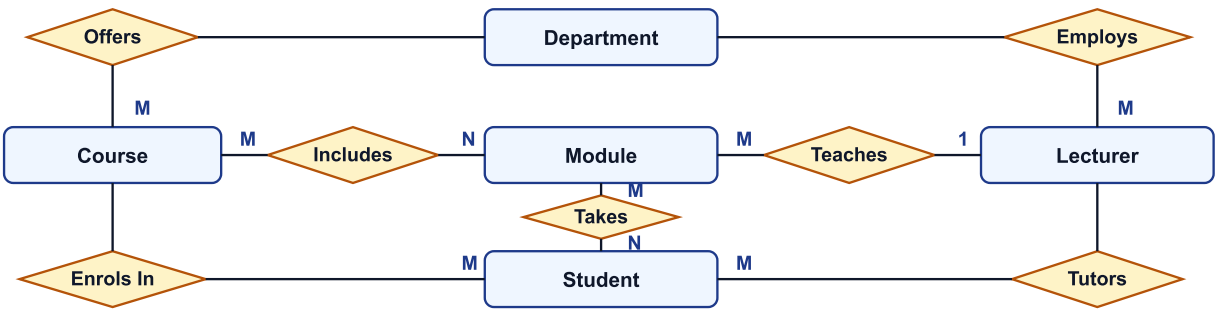


Figure 7: Complete University E/R Diagram with Cardinalities (Lecture 14).

2. Entity vs. Attribute & Eliminating Redundant 1:1 Relationships (Lecture 15)

- **How to decide if a noun is an Entity or an Attribute:**
  1. **General Rule:** Entities have smaller descriptive parts (attributes) and participate in relationships with other entities, whereas an attribute has no smaller parts and belongs to a single entity.
  2. **Multiple Values / Structure Rule (Lec 13 Slide 28):** Should `Address` be an attribute of `Employees` or a separate Entity? If an employee can have **several addresses**, or if we need to query the internal structure (`Street` , `City` , `Postcode`) separately across relationships, `Address` must be modeled as an **Entity**.
- **Merging Redundant 1:1 Relationships (Product-Supplier-Address Example):**
  - Initially, we identify three entities that have smaller parts: `Product (Description, Price)` , `Supplier (Name, Phone number)` , and `Address (Street address, City, Postcode)` .
  - `Product` to `Supplier` is **Many-to-One (M:1)** (many products come from one supplier), so they must stay separate entities.
  - However, `Supplier` to `Address` is **One-to-One (1:1)**, every supplier has one address, and we never store addresses that do not belong to a supplier. Therefore, this `1:1` relationship is **redundant**! We refine the schema by **merging** **`Address`** **into** **`Supplier`** so that `Supplier` holds all five attributes: `Name` , `Phone number` , `Street address` , `City` , and `Postcode` .

3. Classification of Entities in Business Models (Lecture 15, Slide 17)

- **Transactional Entities:** Entities that record measurable business events or transactions that can be summarized (such as *Sales*, *Purchases*, *Orders*, or *Repairs*).
- **Component Entities:** Entities that answer the *Who, What, Where, When, and Why* of a business model:
  - `Customer` : **Who** made the purchase
  - `Product` : **What** was sold
  - `Location` : **Where** it was sold
  - `Period` : **When** it was sold

4. Solved Lectures 14 & 15 Assignments (Theory Questions & Microscope Lab ERD)

- **Q1: What is the major source of information for an ERD, and how is it captured?**
  Answer: The major sources are user requirements specifications, existing business forms/invoices (like the Stereos To Go invoice), interviews, and DFDs. Information is captured during Requirements Analysis by identifying nouns (entities/attributes), verbs (relationships), and business rules (cardinalities/constraints).
- **Q2: How many entities and relationships are mandatory for an ERD? Is an ERD a formal representation, and how is it standardized?**
  Answer: At minimum, an ERD requires at least **one entity** (if it has a unary self-relationship) or **two entities connected by one binary relationship**. Yes, an ERD is a formal conceptual blueprint standardized using formal notations (such as Chen notation or Crow's Foot notation) and refined by merging redundant 1:1 entities and resolving M:N relationships into associative tables.

- **Q3: Solved Design Assignment (Material Science Lab Microscope Reservation System):**
  - **Entities & Attributes:**
    1. `Lab (LabID, LabName, Building)`
    2. `Microscope (MicroscopeID, Model, RequiresAdminPresence)`
    3. `Administrator (AdminID, Name, Email)`
    4. `EndUser (UserID, Name, Email, Department)`
    5. `ReservationRequest (RequestID, RequestDate, Status [Authorized/Denied])`
    6. `TimeSlot (SlotID, SlotDate, StartHour, EndHour)`
  - **Relationships & Cardinalities:**
    - `Lab` (1) **Locates** (M) `Microscope` (Each microscope is located in one specific lab; total participation on Microscope).
    - `EndUser` (1) **Submits** (M) `ReservationRequest` (An end user submits reservation requests).
    - `Microscope` (1) **Is_Reserved_In** (M) `ReservationRequest` (Each request reserves a specific microscope).
    - `Administrator` (1) **Reviews_And_Supervises** (M) `ReservationRequest` (Each request is assigned to one administrator who authorizes/denies it and is present if required).
    - `ReservationRequest` (M) **occupies** (N) `TimeSlot` (Each reservation takes 1 or more 1-hour slots; total participation on ReservationRequest).